BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 7

Arrays Part 1

Table of Contents

SL No	Topic		Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction		-	-	3
	1.1	Objectives	-	-	
2	One Dimensional Arrays		-	-	4-9
	2.1	Passing Arrays To Functions	-	1	
3	Summary		-	-	10
4	Terminal Questions		-	-	11
5	Answers To Self Assessment Questions		-	-	11
6	Answers To Terminal Questions		-	-	11
7	Exercises		-	-	12

1. INTRODUCTION

In the previous unit, you studied about the various types of storage classes that are used in C. You studied how those storage classes are used in different situations in C. In this unit, you will study about the arrays and strings. You will learn how arrays and strings are formed and manipulated.

Many applications require processing of multiple data items that have common characteristics. In such situations it is always convenient to place the data items into an array, where they will share the same name. An array is a collection of similar type of elements. All elements in the array are referred with the array name. Since arrays hold a group of data, it is very easy to perform looping and arithmetic operations on group of data. This chapter covers the processing of both one-dimensional and two-dimensional arrays.

1.1. Objectives:

At studying this unit, you should be able to:

- *Declare, initialize and process one-dimensional*
- *Array Initialization with Declaration*
- *Array Initialization Using Loops*

2. ONE DIMENSIONAL ARRAYS

So far, we've been declaring simple variables: the declaration

```
int i;
```

declares a single variable, named i, of type **int**. It is also possible to declare an array of several elements. The declaration

```
int a[10];
```

declares an array, named a, consisting of ten elements, each of type int. Simply speaking, an array is a variable that can hold more than one value. You specify which of the several values you're referring to at any given time by using a numeric subscript. (Arrays in programming are similar to vectors or matrices in mathematics.) We can represent the array a above with a picture like this:

a:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

In C, arrays are zero-based: the ten elements of a 10-element array are numbered from 0 to 9. The subscript which specifies a single element of an array is simply an integer expression in square brackets. The first element of the array is a[0], the second element is a[1], etc. You can use these “array subscript expressions'' anywhere you can use the name of a simple variable, for example:

```
a[0] = 10;
a[1] = 20;
a[2] = a[0] + a[1];
```

Notice that the subscripted array references (i.e. expressions such as a[0] and a[1]) can appear on either side of the assignment operator.

The subscript does not have to be a constant like 0 or 1; it can be any integral expression. For example, it's common to loop over all elements of an array:

```
int i;
for(i = 0; i < 10; i = i + 1)
        a[i] = 0;
```

This loop sets all ten elements of the array a to 0

Arrays are a real convenience for many problems, but there is not a lot that C will do with them for you automatically. In particular, you can neither set all elements of an array at once nor assign one array to another; both of the assignments

```
a = 0;                          /* WRONG */
```

and

```
int b[10];
b = a;                          /* WRONG */
```

are illegal.

To set all of the elements of an array to some value, you must do so one by one, as in the loop example above. To copy the contents of one array to another, you must again do so one by one:

```
int b[10];
for(i = 0; i < 10; i = i + 1)
b[i] = a[i];
```

Remember that for an array declared

```
int a[10];
```

there is no element a[10]; the topmost element is a[9]. This is one reason that zero-based loops are also common in C. Note that the for loop

```
for(i = 0; i < 10; i = i + 1)
        .....
```

does just what you want in this case: it starts at 0, the number 10 suggests (correctly) that it goes through 10 iterations, but the less-than comparison means that the last trip through the loop has i set to 9. (The comparison i <= 9 would also work, but it would be less clear and therefore poorer style.)

In the little examples so far, we've always looped over all 10 elements of the sample array a. It's common, however, to use an array that's bigger than necessarily needed, and to use a second variable to keep track of how many elements of the array are currently in use. For example, we might have an integer variable

```
int na;          /* number of elements of a[] in use */
```

Then, when we wanted to do something with a (such as print it out), the loop would run from 0 to na, not 10 (or whatever a's size was):

```
for(i = 0; i < na; i = i + 1)
printf("%d\n", a[i]);
```

Naturally, we would have to ensure that na's value was always less than or equal to the number of elements actually declared in a.

Arrays are not limited to type int; you can have arrays of char or double or any other type.

Here is a slightly larger example of the use of arrays. Suppose we want to investigate the behavior of rolling a pair of dice. The total roll can be anywhere from 2 to 12, and we want to count how often each roll comes up. We will use an array to keep track of the counts: a[2] will count how many times we've rolled 2, etc.

We'll simulate the roll of a die by calling C's random number generation function, rand(). Each time you call rand(), it returns a different, pseudo-random integer. The values that rand() returns typically span a large range, so we'll use C's modulus (or "remainder'') operator % to produce random numbers in the range we want. The expression rand() % 6 produces random numbers in the range 0 to 5, and rand() % 6 + 1 produces random numbers in the range 1 to 6.

Program 2.1: Program to simulate the roll of a die

```
#include <stdio.h>
#include <stdlib.h>
main()
{
int i;
int d1, d2;
int a[13];    /* uses [2..12] */

for(i = 2; i <= 12; i = i + 1)
        a[i] = 0;
for(i = 0; i < 100; i = i + 1)
{
        d1 = rand() % 6 + 1;
        d2 = rand() % 6 + 1;
        a[d1 + d2] = a[d1 + d2] + 1;
}
for(i = 2; i <= 12; i = i + 1)
        printf("%d: %d\n", i, a[i]);
return 0;
}
```

We include the header <stdlib.h> because it contains the necessary declarations for the rand() function. We declare the array of size 13 so that its highest element will be a[12]. (We're wasting a[0] and a[1]; this is no great loss.) The variables d1 and d2 contain the rolls of the two individual dice; we add them together to decide which cell of the array to increment, in the line

a[d1 + d2] = a[d1 + d2] + 1;

After 100 rolls, we print the array out. Typically, we'll see mostly 7's, and relatively few 2's and 12's.

2.1. Passing Arrays to Functions

An array name can be used as an argument to a function, thus permitting the entire array to be passed to the function. To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is not specified within the formal argument declaration.

Program 2.2: The following program illustrates the passing of an array from the main to a function. This program is used to find the average of n floating point numbers.

```
#include<stdio.h>
main()
{
        int n, i;
        float avg;
        float list[100];

        float average(int, float[]); /* function prototype */
        printf("How many numbers:");
        scanf("%d",&n);
        printf(" Enter  the numbers:");
        for(i=1;i<=n;i++)
        scanf("%f", &list[i]);
        avg=average(n, list); /* Here list and n are actual arguments */
        printf("Average=%f\n", avg);
```

```
}
float average(int a, float x[ ])
{
        float avg;
        float sum=0;
        int i;
        for(i=0;i<a;i++)
        sum=sum+x[i]; /* find sum of all the numbers */
        avg=sum/a;    /* find average */
        return avg;
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. In C, an array subscript starts from _____
2. An array name is a pointer. (True/False)
3. Will there be a compilation error for the following program segment?(Yes/No)

```
int a[5] = {1, 2, 3, 4, 5};
int b[5] = {5, 4, 3, 2, 1};
int c[5][5];
...
c=a+b;
...
```

4. What is the correct way to declare a 1D array in C?

   (a) int array[10];

   (b) int array[] = {1, 2, 3, 4, 5};

   (c) int array() = {1, 2, 3, 4, 5};

   (d) int array{} = {1, 2, 3, 4, 5};

3. SUMMARY

An array is a variable that can hold more than one value. In C, arrays are zero-based. An array name can be used as an argument to a function, thus permitting the entire array to be passed to the function. The C language allows arrays of any dimension to be defined. Arrays in C are zero-indexed. For example, array [0] accesses the first element. Initialization can be done at declaration using braces {}. Arrays are versatile and commonly used for storing and manipulating data sets, implementing algorithms, and representing sequences. They provide efficient memory usage and easy traversal of elements. However, their size must be known at compile time, and they lack built-in bounds checking, making careful programming essential to avoid accessing out-of-bounds memory.

INSPIRED BY LIFE

4. TERMINAL QUESTIONS

1. Write a program for 10 times summation of square of a number.
2. Write a Program that uses loops for array processing.

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. 0
2. True
3. Yes
4. (a) int array[10];

6. ANSWERS TO TERMINAL QUESTIONS

```
1.  #include<stdio.h>
    main()
    {
        int i=0, sum=0, x;
        printf('Enter a number:");
        scanf("%d", &x);
        while(i<10)
        {
                sum+=x*x;
                i++;
        }
        printf("Sum=%d", sum);
    }

2.  // loops for array processing #include <stdio.h>
    #define SIZE 10
    #define PAR 72
    int main(void)
    {
```

```
int index, score[SIZE]; int sum = 0;
float average;
printf("Enter %d golf scores:\n", SIZE);
for (index = 0; index < SIZE; index++)
{
    scanf("%d", &score[index]); */read in the ten scores
    printf("The scores read in are as follows:\n");
    for (index = 0; index < SIZE; index++)
    printf("%5d", score[index]); */verify input
    printf("\n");
}
for (index = 0; index < SIZE; index++)
{
    sum += score[index];*/add them up
    average = (float) sum / SIZE; */ time-honored method
    printf("Sum of scores = %d, average = %.2f\n", sum, average);
    printf("That's a handicap of %.0f.\n", average - PAR);
    return 0;
}
}
```

7. EXERCISES

1. Write a program to count the number of vowels and consonants in a given string.
2. Write a c program Maximum Element in Array.
3. Write a c program to reverse array elements.